

Perception & State Estimation for Autonomous Drone

Singularit Drone Team, CUSAT

Contents

1	Introduction	2
2	System Modeling	3
2.1	Spatial Reference Frames	3
2.2	Degrees of Freedom (6-DOF)	3
2.3	State Representation	3
2.4	Equations of Motion	4
2.5	Testing and Simulation	4
3	Hardware	5
3.1	Physical Frame & Propulsion	5
3.2	Onboard Compute	5
3.3	Sensor Modalities	6
3.4	Hardware related issues	6
4	Vision	6
4.1	Adaptive Cropping & Preprocessing	8
4.2	Gate Detection Service (YOLO Pose)	8
4.3	Localisation Service (Perspective-n-Point)	8
4.4	Extrinsic Calibration & Outlier Rejection	8
5	IMU Polling & Prediction	9
5.1	Dynamic Model & IMU Saturation Monitor	10
5.2	IMU Polling & Prediction Service (EKF Predict)	10
6	Sensor Fusion & State Update	10
6.1	EKF Update	11

1 Introduction

This document outlines the perception-state estimation pipeline using Visual-Inertial Odometry algorithms for autonomous drones. The perception subsystem expected to receive the following inputs :

1. High-Resolution Raw Image Tensor: $\mathbf{I}_{raw} \in \mathbb{R}^{H_{raw} \times W_{raw} \times 3}$ at 30–90 Hz from the Monocular Camera Driver.
2. Relative 3D Linear Accelerations $\mathbf{a} \in \mathbb{R}^3 = [a_x, a_y, a_z]^T$ and Relative 3D Angular Body Rates $\boldsymbol{\omega}_B \in \mathbb{R}^3 = [p, q, r]^T$ from the IMU at 1,000+ Hz.
3. Pre-loaded Track Flight Plan: A vector of known 3D global gate center coordinates $\mathbf{p}_{W, gate-i} \in \mathbb{R}^3$ defined in the World frame.
4. Known Gate Geometry: The physical 3D dimensions of the gate corners $\mathbf{P}_{G, corners} \in \mathbb{R}^{4 \times 3}$ defined in the local Gate frame.
5. Camera Intrinsic Matrix: The static calibration matrix $K \in \mathbb{R}^{3 \times 3}$ (along with distortion coefficients $\mathbf{D}$) required to mathematically map the 3D gate coordinates into the 2D image plane.
6. Camera-to-Body Extrinsics: The rotational offset matrix $\mathbf{R}_{CB} \in SO(3)$ and translation vector $\mathbf{t}_{CB} \in \mathbb{R}^3$, generated by the offline extrinsic calibration optimizer, required to transform the visual pose into the IMU body frame.

The perception subsystem is designed to output the following state vectors depending on the subsequent planning/control layer.

1. 16D Global State Vector (from EKF Update):

$$\mathbf{X}_{EKF} = [p_W, v_W, q_W, b_a, b_g]^T = [x, y, z, v_x, v_y, v_z, q_w, q_x, q_y, q_z, b_x, b_y, b_z, b_p, b_q, b_r]^T;$$
 - $\mathbf{p}_W, \mathbf{v}_W \in \mathbb{R}^3$: 3D Global position and velocity ie. absolute position of drone's body relative to fixed world map origin (in kinematics p_{WB}, v_{WB})
 - $\mathbf{q}_W \in \mathbb{R}^4$: 4D Global Orientation represented as a unit quaternion.
 - $\mathbf{b}_a, \mathbf{b}_g \in \mathbb{R}^3$: 6D IMU sensor biases tracking the accelerometer drift (b_x, b_y, b_z) and gyroscope drift (b_p, b_q, b_r).
2. 17D Dynamic Global Vector (to MPC):

$$\mathbf{x}_{MPC} = [p_W, \mathbf{q}_W, \mathbf{v}_W, \boldsymbol{\omega}_B, \boldsymbol{\Omega}]^T$$

$$= [x, y, z, q_w, q_x, q_y, q_z, v_x, v_y, v_z, \omega_x, \omega_y, \omega_z, \Omega_1, \Omega_2, \Omega_3, \Omega_4]^T$$
 - $\mathbf{p}_W, \mathbf{q}_W$: The position and orientation of the Body relative to the World (p_{WB}, q_{WB}).
 - $\mathbf{v}_W \in \mathbb{R}^3$: 3D Linear velocity in the world frame.
 - $\boldsymbol{\omega}_B \in \mathbb{R}^3$: 3D Angular body rates across the drone's own axes.
 - $\boldsymbol{\Omega} \in \mathbb{R}^4$: The real-time motor speeds (RPM) of the four individual propellers.
3. 24D Observation Vector (to RL Policy):

$$\mathbf{x}_{obs} = [p_{g_i}, \mathbf{v}_{g_i}, \boldsymbol{\Phi}_{g_i}, \boldsymbol{\Omega}_W, \boldsymbol{\omega}_B, p_{g_{i+1}}, \psi_{g_{i+1}}, \mathbf{u}_{k-1}]^T$$

$$= [x_{g_i}, y_{g_i}, z_{g_i}, v_{x,g_i}, v_{y,g_i}, v_{z,g_i}, \phi_{g_i}, \theta_{g_i}, \psi_{g_i}, \Omega_{x,W}, \Omega_{y,W}, \Omega_{z,W}, \omega_1, \omega_2, \omega_3, \omega_4, x_{g_{i+1}}, y_{g_{i+1}}, z_{g_{i+1}}, \psi_{g_{i+1}}, u_1, u_2, u_3, u_4]^T$$
 - $\mathbf{p}_{g_i}, \mathbf{v}_{g_i} \in \mathbb{R}^3$: The drone's position and velocity inside the *current gate*'s reference frame.
 - $\boldsymbol{\Phi}_{g_i} \in \mathbb{R}^3$: 3D Euler angles (roll ϕ , pitch θ , yaw ψ) replacing the quaternion for the network input.
 - $\boldsymbol{\Omega}_W \in \mathbb{R}^3$: The 3D angular velocity in the world frame.
 - $\boldsymbol{\omega}_B \in \mathbb{R}^4$: The 4D angular velocities of the drone's individual motors.
 - $\mathbf{p}_{g_{i+1}} \in \mathbb{R}^3, \psi_{g_{i+1}} \in \mathbb{R}^1$: The translation vector and yaw angle of the *next* gate, expressed relative to the current gate's frame.
 - $\mathbf{u}_{k-1} \in \mathbb{R}^4$: The 4 individual motor power commands applied in the previous time step.

2 System Modeling

To capture the complex nonlinear dynamics of a quadrotor at the edge of its performance envelope, the vehicle is modeled as a 6 degree-of-freedom (DOF) rigid body of mass m with a diagonal inertia matrix $\mathbf{J} = \text{diag}(J_x, J_y, J_z)$.

2.1 Spatial Reference Frames

Autonomous drone racing architectures rely on multiple coordinate systems to accurately define the relationships between the global track, the moving platform, and local objectives. In high-speed agile flight, it is standard convention to define the following reference frames:

- **World Frame ($\mathcal{F}_W$):** A fixed, global coordinate system used for tracking the drone's position within the map. This is typically defined using a North-East-Down (NED) convention.
- **Body Frame ($\mathcal{F}_B$):** A moving coordinate system physically attached to the drone's center of mass. Sensor measurements such as angular rates and linear accelerations are expressed in this local frame.
- **Camera Frame ($\mathcal{F}_C$):** A local frame attached to the onboard camera lens, tracking the optical axis for vision-based tasks.
- **Gate Frame ($\mathcal{F}_G$):** A local coordinate system attached to the center of the next racing gate, heavily utilized by reinforcement learning policies to calculate relative positionst.

Double subscripts are strictly employed to define the mathematical transformations between these frames. For instance, $\mathbf{p}_{WB}$ denotes the position of the Body frame relative to the fixed World frame, and $\mathbf{q}_{WB}$ defines the unit quaternion rotating the Body frame into the World frame.

2.2 Degrees of Freedom (6-DOF)

In the context of agile quadcopter flight, the drone is mathematically modeled as a 6 Degree-of-Freedom (6-DOF) rigid body. These degrees of freedom represent the minimum number of independent physical coordinates required to completely define the spatial state of the vehicle in the three-dimensional World frame.

The 6-DOF system is fundamentally divided into two types of spatial movement:

- **Translation (3-DOF):** The linear movement of the drone's center of mass along the three spatial axes (typically representing movement along the X, Y, and Z axes). This dictates the absolute spatial location of the vehicle and is mathematically represented by the global position vector $\mathbf{p}_{WB} \in \mathbb{R}^3$.
- **Rotation / Attitude (3-DOF):** The angular orientation of the drone's local Body frame relative to the fixed World frame (commonly referred to as roll, pitch, and yaw). While the vehicle possesses exactly 3 physical rotational degrees of freedom, the system's attitude is mathematically represented using a 4-dimensional unit quaternion $\mathbf{q}_{WB} \in SO(3)$. This higher-dimensional representation is used to ensure a completely singularity-free orientation space (avoiding "gimbal lock") during extreme acrobatic racing maneuvers.

Together, these 6 degrees of freedom dictate the core kinematic and dynamic equations of the drone. By taking the time derivatives of these 6 coordinates, we obtain the drone's linear velocity ($\mathbf{v}_W \in \mathbb{R}^3$) and its angular body rates ($\boldsymbol{\omega}_B \in \mathbb{R}^3$), which are combined with the degrees of freedom to build the comprehensive dynamic state vectors required by the EKF and MPC layers.

2.3 State Representation

The full dynamic state of the quadrotor, representing its evolution in time, is commonly defined as a 17-dimensional vector $\mathbf{x} \in \mathbb{R}^{17}$:

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}_{WB} \\ \mathbf{q}_{WB} \\ \mathbf{v}_W \\ \boldsymbol{\omega}_B \\ \boldsymbol{\Omega} \end{bmatrix} \quad (1)$$

Where:

- $\mathbf{p}_{WB} \in \mathbb{R}^3$: The 3D position of the drone's center of mass in the world frame.
- $\mathbf{q}_{WB} \in SO(3)$: A unit quaternion defining the vehicle's attitude (rotation from the body frame to the world frame).
- $\mathbf{v}_W \in \mathbb{R}^3$: The linear velocity of the vehicle in the world frame.
- $\boldsymbol{\omega}_B \in \mathbb{R}^3$: The angular body rates across the quadrotor's own axes.
- $\boldsymbol{\Omega} \in \mathbb{R}^4$: The real-time rotational speeds of the four individual motors.

2.4 Equations of Motion

The continuous-time kinematics and dynamics governing the quadrotor, subject to control inputs $\mathbf{u} \in \mathbb{R}^4$ (motor speed commands) and gravity $\mathbf{g}_W = [0, 0, 9.81]^T$ m/s², are modeled by the following set of differential equations:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{\mathbf{p}}_{WB} \\ \dot{\mathbf{q}}_{WB} \\ \dot{\mathbf{v}}_W \\ \dot{\boldsymbol{\omega}}_B \\ \dot{\boldsymbol{\Omega}} \end{bmatrix} = \begin{bmatrix} \mathbf{v}_W \\ \mathbf{q}_{WB} \odot \begin{bmatrix} 0 \\ \boldsymbol{\omega}_B/2 \end{bmatrix} \\ \frac{1}{m} (\mathbf{q}_{WB} \odot \mathbf{f}_{total}) + \mathbf{g}_W \\ \mathbf{J}^{-1} (\boldsymbol{\tau}_{total} - \boldsymbol{\omega}_B \times \mathbf{J} \boldsymbol{\omega}_B) \\ \frac{1}{\tau_{mot}} (\boldsymbol{\Omega}_{ss} - \boldsymbol{\Omega}) \end{bmatrix} \quad (2)$$

Where:

- $\odot$ denotes the quaternion-vector product, representing the mathematical rotation of a body-frame vector by the attitude quaternion.
- $\mathbf{f}_{total} \in \mathbb{R}^3$ represents the total forces acting on the drone in the body frame, typically the sum of the propeller lift ($\mathbf{f}_{prop}$) and the highly nonlinear aerodynamic drag forces ($\mathbf{f}_{aero}$).
- $\boldsymbol{\tau}_{total} \in \mathbb{R}^3$ represents the total body-frame moments, comprising the torques generated by propeller thrust ($\boldsymbol{\tau}_{prop}$), yaw moments from motor speed changes ($\boldsymbol{\tau}_{mot}$), aerodynamic torques ($\boldsymbol{\tau}_{aero}$), and gyroscopic inertial terms ($\boldsymbol{\tau}_{iner}$).
- τ_{mot} is the time constant of the motor/ESC loop, and $\boldsymbol{\Omega}_{ss}$ represents the steady-state propeller speeds mapped from the control input commands.

2.5 Testing and Simulation

The following open-source datasets, simulation environments, and tools are utilized to benchmark, test, and calibrate the perception and state estimation pipelines.

Datasets:

- **UZH-FPV Drone Racing Dataset:** Contains real-world high-resolution images, event camera data, IMU measurements, and millimeter-accurate motion capture ground truth designed for evaluating visual-inertial odometry (VIO) during agile maneuvers.
- **Blackbird Dataset:** Provides over 10 hours of real and simulated aggressive flight data specifically for testing VIO and SLAM algorithms.
- **TH Drone Racing Dataset:** Contains high-quality perception data from both autonomous and human-piloted flights.

- **NeuroBEM Dataset:** Flight data representing learned aerodynamic forces from highly aggressive maneuvers.
- **AirSim Drone Gate Dataset:** A synthetic computer vision dataset containing images of racing gates generated via the AirSim simulator.

Simulators:

- **Flightmare:** A flexible simulator with photorealistic rendering that supports hardware-in-the-loop simulation for testing vision pipelines.
- **AirSim:** A photorealistic Unreal Engine simulator capable of testing the entire perception and estimation pipeline with high transferability to the real world.
- **FlightGoggles:** A photorealistic rendering engine built in Unity3D paired with C++ dynamic simulation.
- **Aerial Gym:** A GPU-accelerated simulator for generating vast amounts of perception data, including RGB, depth, segmentation, and optical flow.
- **RotorS:** A Gazebo-based simulator for multi-rotor basic dynamic simulation and contact forces.

VIO tools:

- **VID-Fusion:** A robust visual-inertial-dynamics odometry system that integrates error terms related to the drone's transitional dynamics into the VIO backend for accurate external force estimation [1-3].
- **OpenVINS:** An open-source research platform for visual-inertial estimation based on the Multi-State Constraint Kalman Filter (MSCKF). Filter-based methods like this are commonly used in drone racing due to their favorable trade-off between accuracy and computational requirements.
- **VINS-Mono:** A robust and versatile monocular visual-inertial state estimator. Instead of a filter, it relies on fixed-lag smoothing, solving a non-linear optimization problem over a sliding window of recent robot states.
- **ROVIO:** A visual-inertial odometry system that uses a direct EKF-based approach. It tracks the motion of image patches directly on raw pixel intensities, making it particularly robust in featureless scenarios.
- **Kalibr:** A widespread open-source calibration tool used to estimate the static camera intrinsic parameters, as well as the extrinsic spatial and temporal offsets between the camera and the IMU prior to flight.

3 Hardware

3.1 Physical Frame & Propulsion

1. Chassis: The drone is built on a custom, lightweight carbon fiber frame designed specifically for the competition.
2. Weight: total mass of the fully equipped platform is 966 grams.
3. Propulsion: 5.1-inch propellers driven by high-power brushless DC motors, powered by high-discharge lithium-polymer batteries.
4. Visibility: 8 bright orange LED strips mounted on the arms.

3.2 Onboard Compute

1. Companion Computer: GPU equipped NVIDIA Jetson Orin NX (16GB) as the main processing unit.
2. Flight Controller: equipped with an STM32 ARM processor (such as the STM32H743 running at 480 MHz).

3.3 Sensor Modalities

1. Monocular Camera: primary exteroceptive sensor, single, forward-facing Arducam IMX219 CMOS rolling-shutter camera, provides a wide field of view (155° horizontal by 115° vertical, or 175° diagonal), operating at 90 Hz. The camera connects to the compute board via MIPI cables. The camera driver outputs high resolution raw image tensors at 90Hz $I_{raw} \in \mathbb{R}^{H_{raw} \times W_{raw} \times 3}$
2. Inertial Measurement Unit (IMU): Integrated directly into the flight controller, 3D relative linear acceleration data (accelerometer) $a \in \mathbb{R}^3 = [a_x, a_y, a_z]^T$ at 1,000 Hz and relative 3D angular body rates (gyroscope) $w \in \mathbb{R}^3 = [p, q, r]^T$ at 2,000 Hz.
3. ESC Motor Telemetry: eal-time feedback on motor RPMs

3.4 Hardware related issues

1. Electromagnetic Interference by high thrust loads caused the frame and cables to bend, corrupt frames.
2. Wide-angle big image to process - adaptive cropping of images using predicted gate corners from previous EKF Predict output.
3. Motion blur in camera frames - adjust shutterspeeds, train YOLO on conditioned images.
4. IMU drift v/s camera frequency : Both IMU and camera can be solely utilized for state estimation, but Camera operate at a much slower rate than IMU 1000x, and IMU accumulate drift overtime. EKF Update it used to fuse camera generated actual states into IMU expected pose to correct its drift.
5. IMU biases - updated at EKF prediction node using bias out from the EKF Update
6. Vibration effects on 3D IMU angular body rates - fixed using RPM filtering with the 4D motor speeds from ESC
7. IMU Accelerometer Saturation ie hitting acclerometer limit of 16g (7g mx thrust on complex manueves + vibrations) - dynamically swap the IMU data with mathematical drone model predictions during sharp turns.
8. Battery degradation - Use ESC to monitor and dynamically adjust flight path.
9. flexible TPU mounts used to absorb motor vibrations cause the camera's physical angle relative to the drone's body to shift after every crash or repair - extrinsic callibration optimization btw camera and body (IMU) frames. (Visual-Inertial Callibration)

4 Vision

The vision thread operates asynchronously at 30–90 Hz on the Jetson GPU/CPU. Its primary function is to transform high-dimensional raw image data into a low-dimensional metric state measurement to correct the vehicle's kinematic drift.

Vision Pipeline Thread (T_1) Architecture Stack

Global Image, State, and Calibration Inputs

$$\mathbf{I}_{raw} \in \mathbb{R}^{H_{raw} \times W_{raw} \times 3}, \quad \mathbf{x}_{EKF} \in \mathbb{R}^{16}, \quad \mathbf{G}_i \in \mathbb{R}^3, \quad K, \mathbf{D}, \quad \mathbf{R}_{CB}, \mathbf{t}_{CB}$$

$\Downarrow$

Layer 1: Adaptive Cropping & Preprocessing

Input: $\mathbf{I}_{raw}, \mathbf{x}_{EKF}, \mathbf{G}_i, K, \mathbf{R}_{CB}, \mathbf{t}_{CB}$

$$\mathbf{p}_{c,gate} = \mathbf{R}_{CB} \mathbf{R}_{WB}^T (\mathbf{G}_i - \mathbf{p}_{WB}) + \mathbf{t}_{CB}$$

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{z_{c,gate}} K \mathbf{p}_{c,gate}$$

$$\mathbf{I}_{crop} = \text{Crop}(\mathbf{I}_{raw}, u, v, H_{crop}, W_{crop})$$

$$\text{Output: } \mathbf{I}_{crop} \in \mathbb{R}^{H_{crop} \times W_{crop} \times 3}$$

$\Downarrow$

Layer 2: Gate Detection Service (YOLO Pose)

Input: $\mathbf{I}_{crop}$

$$\mathbf{c}_{crop} = \mathcal{F}_\theta(\mathbf{I}_{crop}) \in \mathbb{R}^{4 \times 2}$$

$$\mathbf{c} = \mathbf{c}_{crop} + \begin{bmatrix} u_{offset} & v_{offset} \end{bmatrix}$$

Output: $\mathbf{c} \in \mathbb{R}^{4 \times 2}$ (Full-Frame Image Corner Keypoints)

$\Downarrow$

Layer 3: Localisation Service (Perspective-n-Point)

Input: $\mathbf{c}, \mathbf{P}_{gate}^{local} \in \mathbb{R}^{4 \times 3}, K, \mathbf{D}$

$$\arg \min_{\mathbf{R}_{rel}, \mathbf{t}_{rel}} \sum_{j=1}^4 \left\| \mathbf{c}_j - \text{Project}(K, \mathbf{D}, \mathbf{R}_{rel}, \mathbf{t}_{rel}, \mathbf{P}_{gate,j}^{local}) \right\|^2$$

Output: Camera-to-Gate Relative Transforms $\mathbf{R}_{rel} \in SO(3), \quad \mathbf{t}_{rel} \in \mathbb{R}^3$

$\Downarrow$

Layer 4: Extrinsic Calibration & Outlier Rejection

Input: $\mathbf{R}_{rel}, \mathbf{t}_{rel}, \mathbf{x}_{EKF}, \mathbf{G}_i, \mathbf{R}_{CB}, \mathbf{t}_{CB}$

$$\mathbf{R}_{meas} = \mathbf{R}_{gate}^{world} \mathbf{R}_{rel}^T \mathbf{R}_{CB}$$

$$\mathbf{p}_{meas} = \mathbf{G}_i - \mathbf{R}_{meas} (\mathbf{R}_{CB}^T \mathbf{t}_{rel} + \mathbf{t}_{CB})$$

Filter: Check $D_M([\mathbf{p}_{meas}, \mathbf{q}_{meas}]^T, \mathbf{x}_{EKF}) \leq \gamma_{thresh}$

Output: Validated Global Pose State Update $\mathbf{z}_{meas} = [\mathbf{p}_{meas}, \mathbf{q}_{meas}]^T \in \mathbb{R}^7$

System States & Map Constants:

- $\mathbf{I}_{raw} \in \mathbb{R}^{H_{raw} \times W_{raw} \times 3}$: Raw input image tensor (30–90 Hz).
- $\mathbf{x}_{EKF} \in \mathbb{R}^{16}$: EKF state vector $[\mathbf{p}_{WB}^T \quad \mathbf{v}_W^T \quad \mathbf{q}_{WB}^T \quad \omega_B^T]^T$ (position, velocity, quaternion, body rates).
- $\mathbf{G}_i \in \mathbb{R}^3$: Known world 3D target gate coordinates $[x_i \quad y_i \quad z_i]^T$
- $\mathbf{P}_{gate,j}^{local} \in \mathbb{R}^3$: Prior 3D spatial coordinates of local gate corner j ($j = 1 \dots 4$).

Calibration Matrices:

- $K \in \mathbb{R}^{3 \times 3}$: Camera intrinsic calibration matrix $\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$.
- $\mathbf{D} \in \mathbb{R}^5$: Lens distortion parameters.
- $\mathbf{R}_{CB} \in SO(3), \mathbf{t}_{CB} \in \mathbb{R}^3$: Static camera-to-body rigid extrinsic transformation parameters computed via prior offline batch optimization.

Intermediate Operational Variables:

- $\mathbf{p}_{c,gate} \in \mathbb{R}^3$: Target gate 3D position vector in camera frame ($\mathcal{F}_c$).
- $[u, v]^T$: Projected target gate center 2D pixel coordinates.
- $\mathbf{I}_{crop} \in \mathbb{R}^{H_{crop} \times W_{crop} \times 3}$: Extracted spatial region-of-interest image sub-tensor.
- $\mathbf{c}_{crop} \in \mathbb{R}^{4 \times 2}$: Bounded local 2D pixel corner coordinates from network.
- $\mathbf{c} \in \mathbb{R}^{4 \times 2}$: Restored full-frame image plane gate corner coordinates.
- $\mathbf{R}_{rel} \in SO(3)$, $\mathbf{t}_{rel} \in \mathbb{R}^3$: Relative camera-to-gate 6-DoF transformation parameters generated by the PnP optimizer.

Output Metrics:

- $\mathbf{R}_{meas} \in SO(3)$, $\mathbf{p}_{meas} \in \mathbb{R}^3$: Reconstructed absolute world frame drone pose.
- $\mathbf{z}_{meas} \in \mathbb{R}^7$: Validated timestamped 6-DoF global pose measurement vector $[\mathbf{p}_{meas}^T \quad \mathbf{q}_{meas}^T]^T$.

4.1 Adaptive Cropping & Preprocessing

- **Objective:** Restrict network processing window to a localized Region of Interest (ROI).
- **Global to Camera Transformation:** Map target gate location into 3D camera tracking frame ($\mathcal{F}_c$):

$$\mathbf{p}_{c,gate} = \mathbf{R}_{CB} \mathbf{R}_{WB}^T (\mathbf{G}_i - \mathbf{p}_{WB}) + \mathbf{t}_{CB}$$

- **Pinhole Projection:** Project 3D camera coordinates to image plane to find expected gate center:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{z_{c,gate}} K \mathbf{p}_{c,gate}$$

- **Tensor Cropping:** Slice sub-tensor at center (u, v) with predefined bounding limits:

$$\mathbf{I}_{crop} = \text{Crop}(\mathbf{I}_{raw}, u, v, H_{crop}, W_{crop})$$

4.2 Gate Detection Service (YOLO Pose)

- **Objective:** Predict 2D inner gate corner pixel locations using an embedded neural network.
- **Inference:** Extract local sub-tensor keypoint coordinate matrix via network model mapping $\mathcal{F}_\theta$:

$$\mathbf{c}_{crop} = \mathcal{F}_\theta(\mathbf{I}_{crop}) = [u_j \quad v_j]^T \in \mathbb{R}^{4 \times 2} \quad \text{for } j = 1 \dots 4$$

- **Coordinate Restoration:** Re-map local indices back to full image coordinate reference system dimensions:

$$\mathbf{c} = \mathbf{c}_{crop} + [u_{offset} \quad v_{offset}]$$

4.3 Localisation Service (Perspective-n-Point)

- **Objective:** Reconstruct local 6-DoF spatial transforms from raw image feature targets.
- **Perspective-n-Point:** Resolve camera-to-gate relative parameters $(\mathbf{R}_{rel}, \mathbf{t}_{rel})$ by minimizing geometric reprojection error over corners:

$$\arg \min_{\mathbf{R}_{rel}, \mathbf{t}_{rel}} \sum_{j=1}^4 \left\| \mathbf{c}_j - \text{Project}(K, \mathbf{D}, \mathbf{R}_{rel}, \mathbf{t}_{rel}, \mathbf{P}_{gate,j}^{local}) \right\|^2$$

4.4 Extrinsic Calibration & Outlier Rejection

- **Objective:** Transform relative camera poses into absolute world coordinates via matrix multiplications utilizing pre-calibrated parameter models, filtering tracking anomalies.
- **Note on Offline Optimization:** Due to onboard computational budget constraints, the structural extrinsic transforms are parameterized through highly accurate, resource-intensive offline batch visual-inertial optimization sequences mapping camera to body coordinates across physical limits:

$$\arg \min_{\mathbf{R}_{CB}, \mathbf{t}_{CB}} \sum_k \left\| \mathbf{e}_{vision,k} \right\|_{\Sigma_v}^2 + \left\| \mathbf{e}_{inertial,k} \right\|_{\Sigma_I}^2$$

- **Relative to Global Transformation:** Multiply relative PnP outputs directly with pre-compiled rigid rotation matrices ($\mathbf{R}_{CB}$) and translation offsets ($\mathbf{t}_{CB}$) to compute world metrics:

$$\mathbf{R}_{meas} = \mathbf{R}_{gate}^{world} \mathbf{R}_{rel}^T \mathbf{R}_{CB}$$

$$\mathbf{p}_{meas} = \mathbf{G}_i - \mathbf{R}_{meas}(\mathbf{R}_{CB}^T \mathbf{t}_{rel} + \mathbf{t}_{CB})$$

- **Outlier Validation Check:** Reject inputs drifting past state vector cross-covariance threshold parameter bounds:

$$\text{If } D_M \left(\begin{bmatrix} \mathbf{p}_{meas} \\ \mathbf{q}_{meas} \end{bmatrix}, \mathbf{x}_{EKF} \right) \leq \gamma_{\text{thresh}} \longrightarrow \text{Push accepted } \mathbf{z}_{meas} \text{ to asynchronous update queue.}$$

5 IMU Polling & Prediction

The high-frequency estimation thread operates deterministically at 1,000–2,000 Hz on the Jetson CPU. Its primary function is to continuously propagate the vehicle's kinematics forward in time, minimizing latency gaps between visual update intervals.

High-Frequency IMU & Prediction Thread (T_2) Architecture Stack

Global Sensor and State Inputs

$$\mathbf{a}_{IMU} \in \mathbb{R}^3, \quad \boldsymbol{\omega}_{filt} \in \mathbb{R}^3, \quad \boldsymbol{\Omega} \in \mathbb{R}^4, \quad \mathbf{x}_{EKF} \in \mathbb{R}^{16}$$

↓

Layer 1: Dynamic Model & IMU Saturation Monitor

Input: $\mathbf{a}_{IMU}, \boldsymbol{\Omega}, \mathbf{x}_{EKF}$

$$\mathbf{a}_{model} = \text{DynamicsSimulation}(\boldsymbol{\Omega}, \mathbf{x}_{EKF})$$

$$\mathbf{a}_{filt}^{IMU}[n] = \alpha \mathbf{a}_{filt}^{IMU}[n-1] + (1 - \alpha) \mathbf{a}_{IMU}[n]$$

$$\mathbf{a}_{filt}^{model}[n] = \alpha \mathbf{a}_{filt}^{model}[n-1] + (1 - \alpha) \mathbf{a}_{model}[n]$$

$$\mathbf{a}_{safe} = \begin{cases} \mathbf{a}_{model} & \text{if } \|\mathbf{a}_{filt}^{model} - \mathbf{a}_{filt}^{IMU}\|_2 > \sigma \\ \mathbf{a}_{IMU} & \text{otherwise} \end{cases}$$

Output: $\mathbf{a}_{safe} \in \mathbb{R}^3$

↓

Layer 2: IMU Polling & Prediction Service (EKF Predict)

Input: $\mathbf{a}_{safe}, \boldsymbol{\omega}_{filt}$

$$\tilde{\mathbf{a}} = \mathbf{a}_{safe} - \mathbf{b}_a, \quad \tilde{\boldsymbol{\omega}} = \boldsymbol{\omega}_{filt} - \mathbf{b}_g$$

$$\begin{bmatrix} \dot{\hat{\mathbf{p}}} \\ \dot{\hat{\mathbf{v}}} \\ \dot{\hat{\mathbf{q}}} \end{bmatrix} = \begin{bmatrix} \hat{\mathbf{v}} \\ \mathbf{R}(\hat{\mathbf{q}})\tilde{\mathbf{a}} + \mathbf{g}_w \\ \frac{1}{2}\hat{\mathbf{q}} \odot \begin{bmatrix} 0 \\ \tilde{\boldsymbol{\omega}} \end{bmatrix} \end{bmatrix}$$

$$\hat{\mathbf{x}}_{pred} = \text{Integrate}(\dot{\hat{\mathbf{p}}}, \dot{\hat{\mathbf{v}}}, \dot{\hat{\mathbf{q}}}, \Delta t)$$

$$\text{Output: } \hat{\mathbf{x}}_{pred} = [\hat{\mathbf{p}}^T, \hat{\mathbf{v}}^T, \hat{\mathbf{q}}^T]^T \in \mathbb{R}^{10}$$

System States & Map Constants:

- $\mathbf{x}_{EKF} \in \mathbb{R}^{16}$: Absolute global state vector pulled from shared memory.
- $\mathbf{g}_w \in \mathbb{R}^3$: Gravity vector relative to the absolute world reference frame.

Matrices / Parameters:

- $\alpha \in \mathbb{R}$: First-order exponential moving-average smoothing constant.
- $\sigma \in \mathbb{R}$: Euclidean distance verification limit checking sensor degradation bounds.
- $\Delta t \in \mathbb{R}$: Micro-timestep matching high-frequency prediction thread update rates.

Intermediate Operational Variables:

- $\mathbf{a}_{IMU} \in \mathbb{R}^3$: Raw linear accelerometer reading streaming from the physical IMU driver.
- $\boldsymbol{\omega}_{filt} \in \mathbb{R}^3$: High-frequency vibration-filtered 3D body rates from flight controller.
- $\boldsymbol{\Omega} \in \mathbb{R}^4$: Actuator rotor velocity values gathered from ESC telemetry.

- $\mathbf{a}_{\text{model}} \in \mathbb{R}^3$: Nominal vehicle frame translation calculation modeled from rigid body drag.
- $\mathbf{a}_{\text{filt}}^{IMU}, \mathbf{a}_{\text{filt}}^{model} \in \mathbb{R}^3$: Exponentially smoothed structural acceleration matrices.
- $\mathbf{a}_{\text{safe}} \in \mathbb{R}^3$: Sanitized structural linear acceleration matrix passed to kinematic estimation.
- $\mathbf{b}_a, \mathbf{b}_g \in \mathbb{R}^3$: Linear accelerometer and gyro bias elements tracked inside the EKF state vector.
- $\tilde{\mathbf{a}}, \tilde{\boldsymbol{\omega}} \in \mathbb{R}^3$: Zero-mean bias-corrected inertial acceleration and body angular rate matrices.

Output Metrics:

- $\hat{\mathbf{x}}_{pred} \in \mathbb{R}^{10}$: Propagated state containing expected kinematics position $\hat{\mathbf{p}}$, velocity $\mathbf{v}\hat{\mathbf{f}}\hat{\mathbf{a}}\mathbf{r}\mathbf{e}$, and orientation $\hat{\mathbf{q}}$ matrices.

5.1 Dynamic Model & IMU Saturation Monitor

- **Objective:** Safeguard state estimation tracking divergence by monitoring and replacing corrupted raw sensor inputs.
- **Model-Based Prediction:** Solve forward rigid-body acceleration values derived from ESC speed signals:

$$\mathbf{a}_{\text{model}} = \text{DynamicsSimulation}(\boldsymbol{\Omega}, \mathbf{x}_{\text{EKF}})$$

- **Low-Pass Filtering:** Isolate structural core trajectories away from high-frequency frame vibrations:

$$\mathbf{a}_{\text{filt}}^{IMU}[n] = \alpha \mathbf{a}_{\text{filt}}^{IMU}[n-1] + (1 - \alpha) \mathbf{a}_{\text{IMU}}[n]$$

$$\mathbf{a}_{\text{filt}}^{model}[n] = \alpha \mathbf{a}_{\text{filt}}^{model}[n-1] + (1 - \alpha) \mathbf{a}_{\text{model}}[n]$$

- **Saturation Check & Handoff:** Pivot trusted sensor inputs seamlessly to forward vehicle models if bounds fail:

$$\mathbf{a}_{\text{safe}} = \begin{cases} \mathbf{a}_{\text{model}} & \text{if } \|\mathbf{a}_{\text{filt}}^{model} - \mathbf{a}_{\text{filt}}^{IMU}\|_2 > \sigma \\ \mathbf{a}_{\text{IMU}} & \text{otherwise} \end{cases}$$

5.2 IMU Polling & Prediction Service (EKF Predict)

- **Objective:** Propagate 10-DoF kinematic states across discrete high-rate micro-timesteps between slow camera steps.
- **Bias Compensation:** Correct incoming streams via running calibration vectors to compute zero-mean inputs:

$$\tilde{\mathbf{a}} = \mathbf{a}_{\text{safe}} - \mathbf{b}_a$$

$$\tilde{\boldsymbol{\omega}} = \boldsymbol{\omega}_{\text{filt}} - \mathbf{b}_g$$

- **Kinematic Integration:** Extrapolate derivative kinematics over the target step using Runge-Kutta updates:

$$\begin{bmatrix} \dot{\hat{\mathbf{p}}} \\ \dot{\hat{\mathbf{v}}} \\ \dot{\hat{\mathbf{q}}} \end{bmatrix} = \begin{bmatrix} \hat{\mathbf{v}} \\ \mathbf{R}(\hat{\mathbf{q}})\tilde{\mathbf{a}} + \mathbf{g}_w \\ \frac{1}{2}\hat{\mathbf{q}} \odot \begin{bmatrix} 0 \\ \tilde{\boldsymbol{\omega}} \end{bmatrix} \end{bmatrix}$$

- **Shared Memory Publication:** Commit state vector $\hat{\mathbf{x}}_{pred}$ into the global update interface for peripheral subsystem lookup.

6 Sensor Fusion & State Update

The sensor fusion update thread operates asynchronously at 30–90 Hz on the Jetson CPU. It runs as a purely event-driven architecture, triggering exclusively upon the arrival of a timestamped global pose measurement from the vision pipeline queue to bound cumulative inertial integration errors.

Event-Driven EKF Update Thread (T_3) Architecture Stack

Global Queue and State Inputs

$$\hat{\mathbf{x}}_{pred} \in \mathbb{R}^{10}, \quad \mathbf{z}_{meas} \in \mathbb{R}^7, \quad \mathbf{P} \in \mathbb{R}^{15 \times 15}, \quad \hat{\mathbf{b}}_{prev} \in \mathbb{R}^6$$

↓

Sensor Fusion Service / EKF Update

Input: $\hat{\mathbf{x}}_{pred}, \mathbf{z}_{meas}, \mathbf{P}, \hat{\mathbf{b}}_{prev}$

$$\hat{\mathbf{x}}_{pred}(t_{cam}) = \text{DelayCompensation}(\hat{\mathbf{x}}_{pred}, t_{cam})$$

$$\mathbf{y} = \mathbf{z}_{meas} - h(\hat{\mathbf{x}}_{pred}(t_{cam}))$$

$$\mathbf{K} = \mathbf{P}\mathbf{H}^T(\mathbf{H}\mathbf{P}\mathbf{H}^T + \mathbf{R})^{-1}$$

$$\mathbf{x}_{EKF} = \begin{bmatrix} \hat{\mathbf{x}}_{pred}(t_{cam}) \\ \hat{\mathbf{b}}_{prev} \end{bmatrix} + \mathbf{K}\mathbf{y}$$

$$\mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}$$

$$\text{Output: } \mathbf{x}_{EKF} = [\mathbf{p}^T, \mathbf{v}^T, \mathbf{q}^T, \mathbf{b}_a^T, \mathbf{b}_g^T]^T \in \mathbb{R}^{16}$$

System States & Map Constants:

- $\mathbf{x}_{EKF} \in \mathbb{R}^{16}$: Absolute global 16-DoF state vector published to shared memory.

Matrices / Parameters:

- $\mathbf{P} \in \mathbb{R}^{15 \times 15}$: Error state covariance matrix tracking system uncertainty profiles.
- $\mathbf{H} \in \mathbb{R}^{6 \times 15}$: Measurement Jacobian matrix mapping error state spaces to pose measurements.
- $\mathbf{R} \in \mathbb{R}^{6 \times 6}$: Measurement noise covariance matrix derived from YOLO-Pose spatial corner uncertainty.
- $\mathbf{I} \in \mathbb{R}^{15 \times 15}$: Standard identity matrix.

Intermediate Operational Variables:

- $\hat{\mathbf{x}}_{pred} \in \mathbb{R}^{10}$: Multi-frequency kinematic state history buffer managed by the T_2 thread.
- $\mathbf{z}_{meas} \in \mathbb{R}^7$: Unsynchronized global pose measurement $[\mathbf{p}_{meas}^T \quad \mathbf{q}_{meas}^T]^T$ from the T_1 thread.
- t_{cam} : Precise hardware timestamp mapped to the visual sensor exposure interval.
- $\hat{\mathbf{x}}_{pred}(t_{cam})$: Time-aligned historical kinematic state retrieved at the matching camera timestamp.
- $h(\cdot) \in \mathbb{R}^7$: Nonlinear measurement model mapping kinematic states to full 6-DoF absolute pose vectors.
- $\mathbf{y} \in \mathbb{R}^6$: Measurement innovation residual vector defined in the system error state space.
- $\mathbf{K} \in \mathbb{R}^{15 \times 6}$: Optimal Kalman gain matrix minimizing error variance.
- $\hat{\mathbf{b}}_{prev} \in \mathbb{R}^6$: Running bias estimation array $[\mathbf{b}_a^T \quad \mathbf{b}_g^T]^T$ composed of linear accelerometer and gyroscope drifts.

Output Metrics:

- $\mathbf{b}_a, \mathbf{b}_g \in \mathbb{R}^3$: Extracted zero-mean compensation biases for downstream T_2 thread hardware correction.
- $[\mathbf{p}^T, \mathbf{v}^T, \mathbf{q}^T]^T$: Validated actual global kinematics dispatched directly to the T_1 cropping loop and T_4 controller.

6.1 EKF Update

- **Objective:** Fuse visual tracking pose vectors with historical odometry logs to correct drift and update internal sensor calibrations.
- **Delay Compensation:** Query the ring buffer to lock the specific historical kinematics matched to target visual data clocks:

$$\hat{\mathbf{x}}_{pred}(t_{cam}) = \text{DelayCompensation}(\hat{\mathbf{x}}_{pred}, t_{cam})$$

- **Kalman Innovation (Residual):** Compute structural tracking deviation values across the observation matrix plane:

$$\mathbf{y} = \mathbf{z}_{meas} - h(\hat{\mathbf{x}}_{pred}(t_{cam}))$$

- **Kalman Gain Computation:** Solve the linear algebraic weight matrix balancing sensor uncertainties with model constraints:

$$\mathbf{K} = \mathbf{P}\mathbf{H}^T(\mathbf{H}\mathbf{P}\mathbf{H}^T + \mathbf{R})^{-1}$$

- **State and Covariance Update:** Wipe out cumulative spatial tracking drifts and adjust persistent sensor bias variables:

$$\mathbf{x}_{\text{EKF}} = \begin{bmatrix} \hat{\mathbf{x}}_{pred}(t_{cam}) \\ \hat{\mathbf{b}}_{prev} \end{bmatrix} + \mathbf{K}\mathbf{y}$$

$$\mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}$$

- **Data Dispatch:** Publish updated global state arrays into the master concurrent shared memory environment.